STUDYSPOT

Campus Study Space Availability Board

Complete Project Walkthrough, Code Exploration & Viva / Presentation Preparation Guide

Document Purpose: A comprehensive preparation guide for all 5 college project team members. Every member must deeply understand their assigned technical module, trace live execution from UI to MongoDB, and master overall MERN architecture, JWT security, and testing.

Project Team Structure & Core Responsibilities

Member Name	Designated Role	Primary Modules	Key Source Files
Varsha Chukka (2303A52494@sru.edu.in)	Member 1: Database & Backend Setup Lead	Express server startup, MongoDB connection, Mongoose model coordination, idempotent seed script.	server/server.js server/app.js server/src/config/db.js server/scripts/seed.js
Syed Abdul Mateen (Maaz) (syedabdulmateen62005@gmail.com)	Member 2: Auth & Security Developer	Registration, login, bcrypt (12 rounds), JWT issuance/verify, transactional moderator invites, auth middleware.	server/src/models/User.js ModeratorInvitation.js authController.js authService.js authMiddleware.js
Kranthi Chint (kranthichint2004@gamil.com)	Member 3: Study Space API Developer	StudySpace schema & compound indexes, multi-filter search aggregation, moderator CRUD, soft deactivation.	server/src/models/StudySpace.js spaceController.js spaceRoutes.js tests/spaces.test.js
Arun Bhuddewar (arunbhuddewar@gmail.com)	Member 4: Availability Reporting Developer	SpaceReport schema, report submission, history pagination, runtime freshness calculation, safe reporter population.	server/src/models/SpaceReport.js reportController.js statusService.js tests/reports.test.js
Shaettha Kownain (2303A52503@sru.edu.in)	Member 5: Frontend Integration & Testing Lead	React 18/Vite SPA, Axios client, AuthContext, route guards, catalog & dashboard UI, payload fix, QA audit report.	client/src/App.jsx client/src/api/* client/src/pages/* QA-REPORT-2026-11-03.md

Recommended Presentation Sequence (12–15 Minutes)

Order	Speaker	Topic & Modules	Time
1	Shaettha (M5)	Problem statement, project vision, UI walkthrough, live catalog, responsive layout.	2.5 min
2	Varsha (M1)	Server setup, MongoDB Atlas connection, model coordination, idempotent seed script.	2.5 min
3	Maaz (M2)	User registration, login, bcrypt password security, JWTs, auth middleware, invites.	3.0 min
4	Kranthi (M3)	StudySpace schema, compound indexing, multi-filter search, moderator CRUD, soft-delete.	2.5 min
5	Arun (M4)	SpaceReport schema, report submission, history pagination, runtime freshness calculation.	2.5 min
Team	All Members	Live end-to-end demonstration, test evidence, and panel viva cross-examination.	2.0 min

1. Project Overview — What StudySpot Solves

StudySpot is a full-stack MERN web application engineered to eliminate the daily frustration students experience searching for quiet, well-equipped campus study spaces. It provides a centralized catalog of campus libraries and reading halls, coupled with peer-to-peer crowdsourced availability reporting and dynamic freshness tracking.

1.1 Problem Statement & Objectives

College students waste valuable study time wandering between academic blocks only to find reading rooms overcrowded or lacking power outlets. Word-of-mouth messages are unverified and quickly become stale. StudySpot provides a centralized, searchable catalog, peer-reported availability updates, runtime freshness calculation, and administrative space governance.

1.2 What the Product Is — And What It Is Not

Implemented & Verified in Codebase	Explicitly NOT Implemented (Honest Scope)
Community-reported availability updates by authenticated students.	Hardware IoT occupancy sensors, cameras, or automated desk counters.
Separation of availability status from report freshness age.	Seat reservations, table bookings, or ticketing algorithms.
Bcrypt password hashing (12 rounds) and 1-day stateless JWTs.	Public self-assignment of moderator roles.
Soft-deactivation of spaces (active: false) preserving all past reports.	Permanent physical deletion of space or report records.
Multi-parameter search: keywords, building, quiet zone, facilities.	Indoor campus GPS navigation or 3D floor map renders.

2. Technical Architecture & Request Lifecycle

StudySpot is structured into decoupled tiers: React 18 Single Page Application (frontend), Express/Node.js REST API (backend), and MongoDB with Mongoose ODM (database).

```
Browser (React 18 SPA) <---> Axios Client (Bearer Token Interceptor)
       |
       v HTTP JSON REST Requests (Port 5000)
Express API (Node.js) -> CORS -> express.json() -> authMiddleware
       |
       +---> /api/auth    ---> authController  ---> User Model
       +---> /api/spaces  ---> spaceController ---> StudySpace Model
       +---> /api/reports ---> reportCtrl/svc  ---> SpaceReport Model
       |
       v Mongoose ODM Queries
MongoDB Atlas (Collections: users, studyspaces, spacereports, moderatorinvitations)
```

■ **Security Boundary Notice:** Frontend route guards (ProtectedRoute, ModeratorRoute) enhance user experience. Real security is strictly enforced on the server by authMiddleware.authenticate and authorizeRoles.

3. Database Design & Schemas

The database schema comprises four normalized collections in MongoDB:

- **User:** name, unique email, hidden passwordHash (select: false), role ('member' | 'moderator').

- **StudySpace:** name, building, area, description, facilities array, quietZone, groupStudyAllowed, active (soft-delete flag), compound index on { active, building, area }.
- **SpaceReport:** spaceId (FK), reporterId (FK), status enum ('Available', 'Moderately Busy', 'Busy', 'Temporarily Closed'), note (<=300 chars), reportedAt, compound index on { spaceId, reportedAt: -1 }.
- **ModeratorInvitation:** tokenHash (SHA-256), expiresAt (TTL index: 24h), usedAt timestamp.

4. Member 1 — Varsha Chukka: Database & Backend Setup Lead

Varsha established server/server.js, server/app.js, and server/src/config/db.js. She implemented hierarchical configuration loading from protected OS appdata, CORS origin whitelisting, central error handling, and the idempotent database seeder (server/scripts/seed.js) which upserts the 5 sample spaces without duplicates.

■ **Member 1 Speaking Script:** "Good morning. I am Varsha Chukka, Member 1 and Database Lead. I implemented server.js, app.js, and db.js. Our server loads private credentials hierarchically to keep secrets out of Git. In app.js, I configured CORS and global error handling. I also created our idempotent database seeder, which populates our 5 campus spaces using deterministic ObjectIds so repeated runs never produce duplicates. Member 2 Maaz will now explain our auth layer."

5. Member 2 — Syed Abdul Mateen (Maaz): Auth & Security Developer

Maaz implemented User.js, authController.js, authService.js, and authMiddleware.js. He enforced bcrypt password hashing (12 salt rounds), stateless 1-day JWT issuance and verification, safe current-user lookup via User.findById(), role-based authorization (authorizeRoles), and secure moderator invitation generation and atomic transactional consumption.

■ **Member 2 Speaking Script:** "Good morning examiners. I am Syed Abdul Mateen, Member 2. I built our authentication and security layer. Passwords are never stored in plaintext—they are hashed with bcrypt using 12 salt rounds. When users log in, we issue a 1-day JWT. Our authMiddleware validates tokens and reloads the user from MongoDB to ensure accounts still exist. I also engineered our single-use moderator invitation system using crypto tokens and MongoDB transactions. Member 3 Kranthi will now present our Space APIs."

6. Member 3 — Kranthi Chint: Study Space API Developer

Kranthi designed StudySpace.js, spaceController.js, and spaceRoutes.js. He engineered our multi-parameter search engine supporting regex text queries and MongoDB aggregation on SpaceReport for availability filtering. He implemented moderator CRUD with strict payload validation and soft-deactivation (active: false) preserving all historical reports.

■ **Member 3 Speaking Script:** "Good morning panel. I am Kranthi Chint, Member 3. I developed our study spaces domain and REST APIs. In StudySpace.js, I created compound indexes on active status, building, and area for fast lookups. In spaceController.js, I built getSpaces() supporting combined filters for building, quiet zones, and amenities. When filtering by availability, my controller runs an aggregation pipeline over SpaceReport. For moderators, DELETE performs a soft deactivation, hiding the space while keeping all reports intact. Member 4 Arun will now explain availability reporting."

7. Member 4 — Arun Bhuddewar: Availability Reporting & Freshness Developer

Arun built SpaceReport.js, reportController.js, and statusService.js. He created authenticated report submission with safe reporter population (only name is returned), newest-first history pagination, and the dynamic freshness calculation service comparing elapsed minutes against STATUS_STALE_AFTER_MINUTES (default 60). Stale reports retain their original availability status.

■ **Member 4 Speaking Script:** "Good morning. I am Arun Bhuddewar, Member 4. I developed our availability reporting and freshness engine. In SpaceReport.js, reports record spaceId, reporterId, status, and timestamp. In reportController.js, report submission derives the student's identity directly from req.user, preventing spoofing. In statusService.js, freshness is calculated at runtime: if elapsed time exceeds 60 minutes, it is marked 'Stale'. Crucially, becoming stale never overwrites availability. Member 5 Shaettha will now present our frontend integration."

8. Member 5 — Shaettha Kownain: Frontend Integration & Testing Lead

Shaettha created the React 18/Vite SPA, Axios client (axios.js), API adapters, AuthContext, route guards (ProtectedRoute, ModeratorRoute), all pages, and the Moderator Dashboard. She resolved a live QA space-edit payload defect, authored 17 passing Vitest tests, and coordinated end-to-end QA.

■ **Member 5 Speaking Script:** "Good morning professors. I am Shaettha Kownain, Member 5. I led our frontend integration and testing. In axios.js, I built our request interceptor that injects Bearer JWTs and a response interceptor handling 401 logouts. In AuthContext.jsx, I created our session restoration provider. I designed all screens including SpacesPage.jsx and ModeratorDashboard.jsx. During live QA, I fixed a space-edit bug by stripping server-managed fields like id and timestamps, and wrote a regression test in moderator-space-edit.test.jsx. Our 58 automated tests pass 100%. We are ready for live demo."

9. Complete Workflows & Demonstration Guide

The system implements 5 verified workflows: (1) Student Member Registration; (2) User Login & Session Restoration; (3) Directory Browsing & Multi-Filter Querying; (4) Availability Report Submission & Dynamic Freshness; and (5) Moderator Space Management & Soft Deactivation.

Live Demonstration Steps (12 Actions):

1. Open http://localhost:5173/ and demonstrate landing page hero visual.
2. Navigate to /spaces; show 5 active spaces loaded from MongoDB via Express.
3. Search 'Library' and check 'Wi-Fi' to show dynamic multi-parameter filtering.
4. Open Central Library Reading Hall; show facilities chips, latest report, and history.
5. Click 'Create account'; register student.demo@sru.edu.in with password StudySpot123!.
6. Verify redirect to /spaces and user name in Navbar.
7. Reopen Central Library; select 'Available', type note, submit report.
8. Verify immediate update: report appears at top of history with 'Fresh' badge.
9. Log out. Log in as moderator@sru.edu.in; verify 'Manage' portal in Navbar.
10. Open /moderator; review metrics bar (active, reported, stale counts).
11. Click 'Add space', create QA space, verify new row appears in table.
12. Click 'Deactivate', confirm in modal; verify soft-deactivation (active: false) and that past reports remain in database.

10. Key Viva Questions & Core Architectural Decisions

Q: Why MERN architecture?
A: Single language (JavaScript) across full stack, reactive UI with React, event-driven REST API with Node/Express, and JSON-like document querying with MongoDB.

Q: Why bcrypt with 12 rounds?
A: Adaptive key-derivation function with automated salt and heavy work factor defeats rainbow tables and GPU brute-force cracking.

Q: Why 1-day JWT?
A: Stateless session authorization that horizontal scales without server RAM overhead. 1-day expiration balances user convenience with security.

Q: Why runtime freshness calculation?
A: Stateless, zero-overhead, always 100% accurate on read. Avoids running continuous database cron jobs.

Q: Why soft deactivation?
A: Preserves historical SpaceReport records and referential integrity while hiding inactive spaces from student discovery.

Q: Why sessionStorage for tokens?
A: Automatically clears when browser tab closes, preventing credential leaks on shared college library computers.

Q: How did you test the backend?
A: Using mongodb-memory-server and Supertest. 41 tests run against isolated in-memory MongoDB without touching live data.

11. Verification Checklist & Final Readiness

- **Backend Tests:** 41 passing tests (server/tests/auth, reports, spaces, seed).
- **Frontend Tests:** 17 passing tests (client/src/test/). Verified 100% pass.
- **Frontend Build:** npm run build in client/ passes cleanly.
- **Live Database:** Atlas connected, 5 spaces seeded, QA space deactivated, reports intact.
- **Demo Accounts:** student.demo@sru.edu.in and moderator@sru.edu.in (StudySpot123!).

StudySpot is complete, verified against actual code, and ready for your college presentation.